

同济大学计算机系

计算机组成原理实验报告



学 号 2452769

姓 名 幸可函

专 业 计算机科学与技术

授课老师 张冬冬

一、实验内容

本实验使用 Verilog 设计并实现 32 位无符号乘法器和 32 位带符号乘法器，分别完成两个 32 位数相乘并输出 64 位结果。实验要求理解乘法器实现原理，完成模块编写与仿真测试，且不允许使用行为级乘号 * 直接实现。

二、硬件逻辑图

略。

三、模块建模

(1) 无符号乘法器

```
module MULTU(  
    input clk,  
    input reset,  
    input [31:0] a,  
    input [31:0] b,  
    output [63:0] z  
);  
    reg [63:0] stored [0:31];  
    reg [63:0] sum2 [0:15];  
    reg [63:0] sum4 [0:7];  
    reg [63:0] sum8 [0:3];  
    reg [63:0] sum16 [0:1];  
    reg [63:0] temp;  
  
    integer i;  
  
    always @(posedge clk or posedge reset) begin  
        if (reset) begin  
            temp <= 64'b0;  
  
            for (i = 0; i < 32; i = i + 1)  
                stored[i] <= 64'b0;  
  
            for (i = 0; i < 16; i = i + 1)  
                sum2[i] <= 64'b0;  
  
            for (i = 0; i < 8; i = i + 1)  
                sum4[i] <= 64'b0;  
  
            for (i = 0; i < 4; i = i + 1)
```

```

        sum8[i] <= 64'b0;

        for (i = 0; i < 2; i = i + 1)
            sum16[i] <= 64'b0;
    end
    else begin
        for (i = 0; i < 32; i = i + 1)
            stored[i] <= b[i] ? ({32'b0, a} << i) : 64'b0;

        for (i = 0; i < 16; i = i + 1)
            sum2[i] <= stored[2 * i] + stored[2 * i + 1];

        for (i = 0; i < 8; i = i + 1)
            sum4[i] <= sum2[2 * i] + sum2[2 * i + 1];

        for (i = 0; i < 4; i = i + 1)
            sum8[i] <= sum4[2 * i] + sum4[2 * i + 1];

        for (i = 0; i < 2; i = i + 1)
            sum16[i] <= sum8[2 * i] + sum8[2 * i + 1];

        temp <= sum16[0] + sum16[1];
    end
end

assign z = temp;
endmodule

```

该模块实现两个 32 位无符号数的乘法运算，输出 64 位无符号乘积。设计采用指导书给出的“部分积分层生成并相加”思路：先根据乘数 b 的各位生成 32 个部分积，再通过多级加法逐级求和，最终得到乘法结果。

(2) 有符号乘法器

```

module MULT(
    input clk,
    input reset,
    input [31:0] a,
    input [31:0] b,
    output [63:0] z
);
    reg [63:0] stored [0:31];
    reg [63:0] sum2 [0:15];
    reg [63:0] sum4 [0:7];

```

```

reg [63:0] sum8 [0:3];
reg [63:0] sum16 [0:1];
reg [63:0] temp;

reg sign1;
reg sign2;
reg sign3;
reg sign4;
reg sign5;

wire sign;
wire [31:0] abs_a;
wire [31:0] abs_b;
wire [63:0] unsigned_result;

integer i;

assign sign = a[31] ^ b[31];
assign abs_a = a[31] ? (~a + 1'b1) : a;
assign abs_b = b[31] ? (~b + 1'b1) : b;
assign unsigned_result = sum16[0] + sum16[1];

always @(posedge clk or posedge reset) begin
    if (reset) begin
        temp <= 64'b0;
        sign1 <= 1'b0;
        sign2 <= 1'b0;
        sign3 <= 1'b0;
        sign4 <= 1'b0;
        sign5 <= 1'b0;

        for (i = 0; i < 32; i = i + 1)
            stored[i] <= 64'b0;

        for (i = 0; i < 16; i = i + 1)
            sum2[i] <= 64'b0;

        for (i = 0; i < 8; i = i + 1)
            sum4[i] <= 64'b0;

        for (i = 0; i < 4; i = i + 1)
            sum8[i] <= 64'b0;

        for (i = 0; i < 2; i = i + 1)

```

```

        sum16[i] <= 64'b0;
    end
    else begin
        sign1 <= sign;
        sign2 <= sign1;
        sign3 <= sign2;
        sign4 <= sign3;
        sign5 <= sign4;

        for (i = 0; i < 32; i = i + 1)
            stored[i] <= abs_b[i] ? ({32'b0, abs_a} << i) : 64'b0;

        for (i = 0; i < 16; i = i + 1)
            sum2[i] <= stored[2 * i] + stored[2 * i + 1];

        for (i = 0; i < 8; i = i + 1)
            sum4[i] <= sum2[2 * i] + sum2[2 * i + 1];

        for (i = 0; i < 4; i = i + 1)
            sum8[i] <= sum4[2 * i] + sum4[2 * i + 1];

        for (i = 0; i < 2; i = i + 1)
            sum16[i] <= sum8[2 * i] + sum8[2 * i + 1];

        temp <= sign5 ? (~unsigned_result + 1'b1) : unsigned_result;
    end
end

assign z = temp;

endmodule

```

该模块实现两 32 位带符号数的乘法运算，输出 64 位带符号乘积。模块先根据输入的符号位判断结果符号，并分别求出两个操作数的绝对值；随后复用与无符号乘法器相同的“部分积生成 + 分层相加”结构完成绝对值相乘；最后再依据符号对乘积进行修正，得到最终带符号结果。

四、测试模块建模

(1) 无符号乘法器

```

module MULTU_tb();
    reg clk;
    reg reset;
    reg [31:0] a;

```

```

reg [31:0] b;
wire [63:0] z;

MULTU uut (
    .clk(clk),
    .reset(reset),
    .a(a),
    .b(b),
    .z(z)
);

initial begin
    clk = 0;
    forever #2 clk = ~clk;
end

initial begin
    reset = 1;
    a = 32'b0;
    b = 32'b0;

    #20;
    reset = 0;

    a = 32'h00000000;
    b = 32'h00000000;

    #50;
    a = 32'h00000000;
    b = 32'hFFFFFFFF;

    #50;
    a = 32'hFFFFFFFF;
    b = 32'h00000000;

    #50;
    a = 32'hFFFFFFFF;
    b = 32'hFFFFFFFF;

    #50;
    a = 32'h80000000;
    b = 32'hAAAAAAAA;

    #50;

```

```

a = 32'hAAAAAAAA;
b = 32'h80000000;

#50;
a = 32'h00000001;
b = 32'h00000001;

#50;
a = 32'hFFFFFFFF;
b = 32'h00000001;

#50;
a = 32'h00000001;
b = 32'hFFFFFFFF;

#50;
a = 32'h80000000;
b = 32'h80000000;

#50;
a = 32'h0000002D;
b = 32'h00000068;

#50;
a = 32'h12345678;
b = 32'h87654321;

#50;
a = 32'h55555555;
b = 32'hAAAAAAAA;

#50;
a = 32'h7FFFFFFF;
b = 32'h00000002;

#50;
a = 32'hFFFFFFFE;
b = 32'h00000002;

#50;
a = 32'h00000000;
b = 32'h00000000;

end
endmodule

```

(2) 有符号乘法器

```
module MULT_tb();
    reg clk;
    reg reset;
    reg [31:0] a;
    reg [31:0] b;
    wire [63:0] z;

    MULT uut (
        .clk(clk),
        .reset(reset),
        .a(a),
        .b(b),
        .z(z)
    );

    initial begin
        clk = 0;
        forever #2 clk = ~clk;
    end

    initial begin
        reset = 1;
        a = 32'b0;
        b = 32'b0;

        #20;
        reset = 0;

        a = 32'h00000000;
        b = 32'h00000000;

        #50;
        a = 32'h00000000;
        b = 32'hFFFFFFFF;

        #50;
        a = 32'hFFFFFFFF;
        b = 32'h00000000;

        #50;
        a = 32'hFFFFFFFF;
```

```
b = 32'hFFFFFFFF;
```

```
#50;
```

```
a = 32'h80000000;
```

```
b = 32'hAAAAAAAA;
```

```
#50;
```

```
a = 32'hAAAAAAAA;
```

```
b = 32'h80000000;
```

```
#50;
```

```
a = 32'h00000001;
```

```
b = 32'h00000001;
```

```
#50;
```

```
a = 32'hFFFFFFFF;
```

```
b = 32'h00000001;
```

```
#50;
```

```
a = 32'h00000001;
```

```
b = 32'hFFFFFFFF;
```

```
#50;
```

```
a = 32'h80000000;
```

```
b = 32'h80000000;
```

```
#50;
```

```
a = 32'h0000002D;
```

```
b = 32'h00000068;
```

```
#50;
```

```
a = 32'h12345678;
```

```
b = 32'h87654321;
```

```
#50;
```

```
a = 32'h55555555;
```

```
b = 32'hAAAAAAAA;
```

```
#50;
```

```
a = 32'h7FFFFFFF;
```

```
b = 32'h00000002;
```

```
#50;
```

```
a = 32'hFFFFFFFE;
```

```

        b = 32'h00000002;

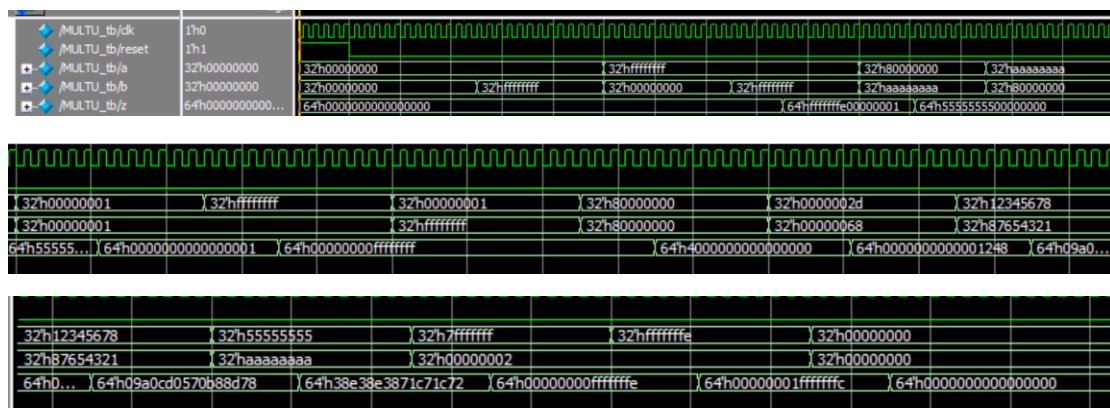
#50;
        a = 32'h00000000;
        b = 32'h00000000;
    end

endmodule

```

五、实验结果

(1) 有符号乘法器



(2) 有符号乘法器

